Page 1 — Project Overview and Test Preparation

VYBE | QA walkthrough | Basis: checked repository routes, UI source, and API handlers | Environment details not supplied

Purpose and scope: VYBE is a social application with community content and interest areas. Validate sign-in, content interactions, discovery, messaging, profile/settings, career, and football features. This walkthrough guides manual functional, UI, API, security, and regression checks; it is not evidence that these checks have already passed.

Modules identified

Public landing, registration/login; authenticated feed and stories; create post; explore/trending; profile/follow/highlights; saved posts; messages; notifications; settings (account, security, privacy, language, accessibility, appearance, blocked users, help); career (jobs/profiles); sports (fixtures, standings, news, clubs/fan posts).

Objectives and setup

- Verify critical user journeys and persistence across refresh/re-login.
- Check required fields, invalid inputs, loading/empty/error states, duplicate actions, and ownership/privacy boundaries.
- Compare the UI with saved records and documented API responses; record browser, OS, build/commit, timestamp, and network conditions.
- Use the project's local start instructions: run `npm run dev`, open `http://localhost:3000`. For shared QA, obtain the deployed base URL and release identifier.

Prerequisites / test data

- Configured Supabase project and app environment variables; valid `FOOTBALL_DATA_API_KEY` for fixtures/standings. Do not copy secrets into reports.
- Two authorized test users (A/B) with confirmed email; no test credentials are present. Confirm intended roles—no distinct role/permission matrix is documented.
- Seed safe test posts, image/video files (including 50MB boundary files), story, job, club, and messages. Use non-sensitive data.
- Have an API client (Postman/curl) and authorized test environment. No Swagger/OpenAPI file or database migrations/RLS definitions were found.

Readiness checklist

- □ Base URL/build and browser recorded
- □ Email confirmation access available
- □ Test media/data prepared and approved
- □ Supabase connected; test users ready
- □ Football API key/quota confirmed
- □ Reset/cleanup method and RLS policies confirmed

> **Confirm before execution:** credentials/roles, production-like environment, exact data-retention rules, RLS policies, expected notification preferences, and API provider quotas are not in the repository materials.

Page 2 — Navigation and Core Functional Testing

Access: Open the agreed base URL. Use landing-page links for registration or login. After authentication, navigate through the app's main navigation to Feed, Create, Explore, Trending, Profile, Saved, Messages, Notifications, Career, Sports, and Settings. Route names confirmed in the app include `/app`, `/app/create`, `/app/explore`, `/app/trending`, `/app/profile`, `/app/saved`, `/app/messages`, `/app/notifications`, `/app/career`, `/app/sports`, and `/app/settings`.

Test Case ID: AUTH-01 | **Module/Feature:** Registration/login

Test Scenario: Positive registration, email confirmation, then login; negative password mismatch and incorrect login.

Preconditions: Approved unused email; access to its inbox; app configured.

Test Steps: Open Register; submit full name, valid email, password of at least 8 characters, matching confirmation; follow confirmation link if requested; log in. Repeat with mismatch, then invalid credentials.

Test Data: New approved test email; matching and mismatching password pairs; wrong password.

Expected Result: Valid account is created and proceeds to app or confirmation prompt; mismatch is rejected; invalid credentials show an error and do not authenticate.

Actual Result: To be completed during testing. | **Status:** Not Run / Pass / Fail / Blocked | **Priority:** Critical

Test Case ID: FEED-01 | **Module/Feature:** Feed, post interactions, stories

Test Scenario: Create/view content and verify like/comment/save actions persist; reject unsupported/invalid media selection.

Preconditions: Signed in; user A and B; valid test media; cleanup permission.

Test Steps: Create a text post and a post with allowed media; verify feed/profile; like and unlike, comment, save and unsave; refresh. Open a story, view/like/reply where available; verify resulting state.

Test Data: Unique text marker; supported image or video; mixed image+video selection for negative check.

Expected Result: Successful actions show consistent updated state after refresh; unsupported mixed media is rejected per UI behavior; no duplicate action or unrelated record appears.

Actual Result: To be completed during testing. | **Status:** Not Run / Pass / Fail / Blocked | **Priority:** Critical

Test Case ID: NAV-01 | **Module/Feature:** Protected navigation

Test Scenario: Signed-out access to protected app; authenticated access to main sections.

Preconditions: One signed-in and one signed-out browser session.

Test Steps: Sign out; enter `/app` and a protected subroute directly; sign in; navigate each listed module; use browser back/refresh.

Test Data: Valid account; signed-out session.

Expected Result: Signed-out access redirects to login; signed-in user can reach authorized sections without broken route or lost session. Confirm exact intended redirect URL with product owner.

Actual Result: To be completed during testing. | **Status:** Not Run / Pass / Fail / Blocked | **Priority:** High

Capture: route, action, visible message, resulting state, console/network error if any, and screenshot/video with private data obscured.

Page 3 — Module-by-Module Walkthrough

Run each check with user A; use user B for visibility/ownership checks. For every page, check initial loading, empty state, populated state, refresh, and an error/offline state. Search/filter controls are present in Explore, Career, blocked-user list, and sports features; verify results update and clearing restores the original set. Pagination/sorting controls are not established by supplied materials; test only if visible and confirm intended behavior.

Test Case ID: DISC-01 | **Module/Feature:** Explore, profile, follow
Test Scenario: Search/discover profiles, open profile, follow/unfollow; invalid/empty search and duplicate follow action.
Preconditions: A/B profiles with discoverable data; A signed in.
Test Steps: Open Explore; enter a known username and a no-match value; clear search; open B's profile; follow, refresh, unfollow, repeat follow; inspect follower counts and feed/profile visibility.
Test Data: Existing B username; unique no-match query.
Expected Result: Results correspond to query; no-match state is clear; follow state/count is consistent and a repeated click does not create duplicate relationships. Exact private-account behavior needs confirmation.
Actual Result: To be completed during testing. | **Status:** Not Run / Pass / Fail / Blocked | **Priority:** High

Test Case ID: CAREER-01 | **Module/Feature:** Career jobs/profiles
Test Scenario: Browse/search/filter jobs; create/edit/remove own job; attempt to edit another user's job; open application link.
Preconditions: A/B accounts; agreed permissions and seeded job; safe external URL.
Test Steps: Search a known term and no-match term; change job type/location filters; create a job using displayed form; edit and remove A's job; try editing B's job; follow application link.
Test Data: Unique test job title; valid and malformed values per visible fields; B-owned job.
Expected Result: Search/filter results match; own changes persist; UI rejects unauthorized edit/removal (code provides an ownership error); external link opens the configured destination. Required fields and validation limits beyond the rendered form need confirmation.
Actual Result: To be completed during testing. | **Status:** Not Run / Pass / Fail / Blocked | **Priority:** High

Test Case ID: MSG-01 | **Module/Feature:** Messages and notifications
Test Scenario: Send/read a message; verify notification list/preferences and cross-user separation.
Preconditions: A/B accounts; messaging permissions and preference defaults confirmed.
Test Steps: A sends B a unique message; B opens Messages and Notifications; mark/read relevant item if supported; refresh; inspect A's and B's views; toggle one notification preference and reload.
Test Data: Non-sensitive unique message; one preference setting.
Expected Result: Message is delivered once and only to intended participants; notifications reflect activity and saved preference. Exact notification delivery rules need confirmation.
Actual Result: To be completed during testing. | **Status:** Not Run / Pass / Fail / Blocked | **Priority:** High

Other modules to cover

Saved: save/unsave a feed item, refresh, verify list. **Settings:** update account/security/privacy/language/accessibility/appearance; verify persisted values and cancellation/errors. **Blocked users:** search, block/unblock B, verify list and documented blocking effects after owner confirmation. **Sports:** check fixtures, standings, news, club view, fan posts and trend filtering. For every form, submit empty, valid, boundary, and malformed values only against fields actually shown.

Page 4 — API, Integration, and Data Validation

Available routes (GET): `/api/fixtures` accepts optional `competition` and `team`; `competition` must match 2–8 uppercase alphanumeric characters after normalization, `team` maximum 100 characters. It queries Football-data.org matches for yesterday through seven days ahead. `/api/standings` accepts optional `competition`, default PL; allowed codes: PL, CL, PD, BL1, SA, FL1. `/api/sports-news` requires `club` (1–100 chars); optional `league`; sources Google News RSS. `/api/supabase-check` checks Supabase Auth health. No API documentation file was found.

Execution: In authorized non-production environment, request each known route using Postman/curl. Check HTTP status, JSON content type, response shape, UI rendering, and provider failures. Do not submit writes to database through ad hoc requests; no write API contract is documented. Never include keys/tokens in evidence.

Test Case ID: API-01 | **Module/Feature:** Fixtures and standings integration

Test Scenario: Valid, invalid, omitted, and boundary query parameters; provider/config errors.

Preconditions: QA base URL; key configured and quota available for positive case.

Test Steps: GET `/api/fixtures`; repeat with `?competition=PL&team=Arsenal`; test invalid punctuation in competition and team length 101. GET `/api/standings`, then `?competition=PL`, then unsupported code. Compare response data to Sports UI.

Test Data: PL, Arsenal; malformed competition; 101-character team; unsupported standings code.

Expected Result: Valid requests return match/table JSON and UI agrees; invalid fixture values return 400; unsupported standings competition returns 400 with allowed-code information. Missing provider key yields configured error; provider 401/403/429/connection errors are surfaced as documented status/error (fixtures/standings behavior may depend on upstream plan).

Actual Result: To be completed during testing. | **Status:** Not Run / Pass / Fail / Blocked | **Priority:** High

Test Case ID: API-02 | **Module/Feature:** Sports news and Supabase health

Test Scenario: Missing/valid/oversized club query; optional league; health endpoint availability.

Preconditions: QA environment with outbound network.

Test Steps: GET `/api/sports-news`; try `?club=Arsenal&league=Premier%20League`; try club length 101 and empty club; GET `/api/supabase-check`. Compare valid article list with Sports UI.

Test Data: Arsenal; 101-character club; empty value.

Expected Result: Missing/invalid club returns 400; valid request returns JSON with articles/query/update time or documented 502 on provider failure; health check reports Auth health. Incorrect type tests are inapplicable to these string query parameters unless API contract expands.

Actual Result: To be completed during testing. | **Status:** Not Run / Pass / Fail / Blocked | **Priority:** Medium

Test Case ID: DATA-01 | **Module/Feature:** Supabase-backed records and UI consistency

Test Scenario: Confirm creates/interactions are persisted exactly once and scoped to correct user.

Preconditions: Authorized read-only database inspection or project-approved admin tooling; schema and RLS access rules supplied.

Test Steps: Create unique test post; like/comment/save; inspect authorized records; refresh/re-login; delete/undo if supported; inspect cleanup.

Test Data: Unique marker and test users A/B.

Expected Result: Displayed and persisted values agree, no duplicates, and user ownership/privacy rules match approved schema/policies. Blocked until schema/RLS documentation and inspection access are supplied.

Actual Result: To be completed during testing. | **Status:** Not Run / Pass / Fail / Blocked | **Priority:** Critical

Page 5 — Negative Testing, Security, and Edge Cases

Run first against staging/test data. Do not probe production accounts or attempt destructive actions outside approved scope. Mark any policy-dependent expected result **Needs confirmation**.

Test Case ID: SEC-01 | **Module/Feature:** Authentication and authorization

Test Scenario: Unauthenticated route access, session expiry/sign-out, and cross-user record access.

Preconditions: A/B test accounts; approved method to expire session and inspect access; RLS policies supplied.

Test Steps: In signed-out session open `/app` and a protected route; sign in then sign out and reuse browser Back/direct URL; expire session and refresh; as A attempt to view/edit/delete B's private records using app UI and approved API tooling.

Test Data: A/B sessions; B-owned post/job/message.

Expected Result: Protected routes require authentication; expired/signed-out session cannot continue private actions; user A cannot access or mutate B-only data. Exact session expiry and privacy policies need confirmation; missing RLS docs make direct database authorization checks Blocked.

Actual Result: To be completed during testing. | **Status:** Not Run / Pass / Fail / Blocked | **Priority:** Critical

Test Case ID: NEG-01 | **Module/Feature:** Forms, media upload, repeated action

Test Scenario: Empty/invalid inputs, size boundary, mixed media, double-click, network interruption.

Preconditions: Signed-in user; approved disposable content; network throttling/offline controls.

Test Steps: Submit each visible form empty; try email/password constraints; select 50MB and over-50MB media and mixed images/video; double-click submit/like/send; interrupt network during create, then reconnect and inspect state.

Test Data: Empty values, invalid email, 7-character password, mismatched confirmation; media at/over 50MB, mixed types; unique action marker.

Expected Result: Required/validation errors prevent invalid submission; media limit/type rule is enforced (source indicates 50MB and does not allow mixing images and video); repeated/ retried actions create no duplicate; failure is visible and final persisted state is clear. Confirm whether 50MB is inclusive and permitted file formats.

Actual Result: To be completed during testing. | **Status:** Not Run / Pass / Fail / Blocked | **Priority:** High

Test Case ID: EDGE-01 | **Module/Feature:** Reliability and error handling

Test Scenario: Empty results, slow/unavailable services, malformed/large search, stale data.

Preconditions: Staging environment; network tools; provider test plan approved.

Test Steps: Search for no match; load Sports with provider unavailable/quota exhausted; throttle network and navigate/refresh; use 100-character then 101-character club/team values; inspect loading, retry, error, and stale-data behavior.

Test Data: No-match query; exact 100/101-character values; simulated timeout/429 only if safely available.

Expected Result: Empty/error/loading state is understandable; no crash or false success; route validation matches documented limits; retry does not duplicate data. Cache/retry behavior beyond route source needs confirmation.

Actual Result: To be completed during testing. | **Status:** Not Run / Pass / Fail / Blocked | **Priority:** Medium

Security observations to record

- Verify no passwords, access tokens, or private message contents appear in URLs, logs exposed to users, or screenshots.
- Check logout invalidates access and user A cannot see B's private messages/records.
- Confirm uploaded media and profile/story visibility rules, blocking behavior, data retention, and account deletion expectations with the product owner; these policies are not documented here.

Page 6 — Regression, Bug Reporting, and QA Sign-off

Smoke and regression checklist

- □ Landing and key links load
- □ Signed-out protected route is blocked
- □ Like/comment/save persists; no duplicates
- □ Messages and notifications scoped correctly
- □ Sports fixtures/standings/news and errors
- □ Responsive layout and keyboard focus usable
- □ Register, confirm email, login, logout
- □ Feed and stories load; create/view interaction
- □ Explore/profile/follow and search work
- □ Career search and own-job workflow
- □ Settings persist; blocked list works
- □ No console/runtime errors on core journeys

Essential regression test cases

Test Case ID: REG-01 | **Module/Feature:** Authentication + core feed
Test Scenario: Verify login-to-feed-to-logout after a change.
Preconditions: Existing confirmed test account.
Test Steps: Log in; load feed; open/create a test post; interact once; refresh; sign out; attempt protected route.
Test Data: Approved test account and unique marker.
Expected Result: Session, feed and interaction work; state persists once; logout blocks protected route.
Actual Result: To be completed during testing. | **Status:** Not Run / Pass / Fail / Blocked | **Priority:** Critical

Test Case ID: REG-02 | **Module/Feature:** Sports + Career
Test Scenario: Verify key read and write journeys after a change.
Preconditions: Provider key and seeded QA records available.
Test Steps: Load fixture/standings/news; use a search/filter; create and remove a test career job; reload both modules.
Test Data: Known team/competition and uniquely named QA job.
Expected Result: API/UI data agree; filters work; own job changes persist and cleanup succeeds; errors are visible if provider is unavailable.
Actual Result: To be completed during testing. | **Status:** Not Run / Pass / Fail / Blocked | **Priority:** High

Bug report template

Title: [module] concise observed failure **Environment:** URL/build/browser/OS/date **Preconditions:** account/data/state

Steps to reproduce: numbered actions **Actual result:** what happened **Expected result:** documented requirement or mark “needs confirmation”

Severity: Critical / High / Medium / Low **Priority:** Critical / High / Medium / Low **Evidence:** screenshot/video, sanitized console/network details, test case ID

Final pass/fail and outstanding questions

□ Critical cases pass □ High cases pass or have accepted defects □ Defects linked with evidence □ Regression complete
□ Blocked cases listed □ QA owner/date recorded

Open confirmations: test credentials and role matrix; exact privacy/RLS rules and schema; profile/story visibility and blocking effects; upload accepted formats and 50MB boundary; notification preferences/delivery; pagination/sort expectations where present; API key/quota and expected provider outage handling; supported browser/device matrix; release acceptance criteria.